

Тема: ПР 21 Создание собственной библиотеки

Цель: совершенствование навыков составления программ со своей библиотекой

Задачи:

- ✓ повторить структуру алгоритмов решения задач с библиотекой;
- ✓ повторить синтаксис операторов работы с графикой;
- ✓ приобрести навыки составления программ с графикой.

Теоретический материал

Теперь пришло время создать свой модуль(библиотеку). Создадим файл mymodule.py, в которой определим какие-нибудь функции:

```
def hello():

    print('Hello, world!')


def fib(n):

    a = b = 1

    for i in range(n - 2):

        a, b = b, a + b

    return b
```

Теперь в этой же папке создадим другой файл, например, main.py:

```
import mymodule
```

```
mymodule.hello()

print(mymodule.fib(10))
```

Выведет:

```
Hello, world!
```

55

### Как назвать модуль?

Помните, что вы (или другие люди) будут его импортировать и использовать в качестве переменной. Модуль нельзя именовать также, как и ключевое слово (их список можно посмотреть [тут](#)). Также имена модулей нельзя начинать с цифры. И не стоит называть модуль также, как какую-либо из **встроенных функций**. То есть, конечно, можно, но это создаст большие неудобства при его последующем использовании.

### Куда поместить модуль?

Туда, где его потом можно будет найти. Пути поиска модулей указаны в переменной `sys.path`. В него включены текущая директория (то есть модуль можно оставить в папке с основной программой), а также директории, в которых установлен python. Кроме того, переменную `sys.path` можно изменять вручную, что позволяет положить модуль в любое удобное для вас место (главное, не забыть в главной программе модифицировать `sys.path`).

### Можно ли использовать модуль как самостоятельную программу?

Можно. Однако надо помнить, что при импортировании модуля его код выполняется полностью, то есть, если программа что-то печатает, то при её импортировании это будет напечатано. Этого можно избежать, если проверять, запущен ли скрипт как программа, или импортирован. Это можно сделать с помощью переменной `__name__`, которая определена в любой программе, и равна `"__main__"`, если скрипт запущен в качестве главной программы, и имя, если он импортирован. Например, `mymodule.py` может выглядеть вот так:

```
def hello():

    print('Hello, world!')
```

```
def fib(n):  
  
    a = b = 1  
  
    for i in range(n - 2):  
  
        a, b = b, a + b  
  
    return b  
  
if __name__ == "__main__":  
  
    hello()  
  
    for i in range(10):  
  
        print(fib(i))
```

**Практическая часть** Составьте игровую модуль программы (приветствуется оконный вид) по заданной тематике

1 Создайте для модуля **'circle'**: функции circle\_perimeter() – вычисляет длину окружности, circle\_area() – вычисляет площадь окружности.

Еще заведем переменную default\_radius = 5, которая будет скрыта при импорте модуля.

Ее назначение – дефолтный радиус для окружности, если пользователь не введет свой.

Обе функции принимают на вход только радиус.

2 Создайте для модуля **'triangle'**: функции triangle\_perimeter() – вычисляет периметр треугольника, triangle\_area() – вычисляет площадь фигуры.

Дополнительно создадим три переменные (длины сторон треугольника): a = 7, b = 2, c = 8, которые также не будут видны при импорте.

На вход функциям передается длина трех сторон (если пользователь ничего не введет, то используются значения по умолчанию).

3) Создайте для модуля 'square': функции square\_perimeter() – вычисляет периметр квадрата, square\_area() – вычисляет площадь фигуры.

Дополнительная переменная  $a = 15$  не доступна при импорте и принимается функциями, если пользователь не предоставил свои размеры стороны квадрата.

4. Реализовать в виде модуля набор подпрограмм для выполнения следующих операций над комплексными числами:

- 1) сложения;
- 2) вычитания;

Используя этот модуль, решить задачу - дан массив  $a$  — массив комплексных чисел. получить массив  $c$ , элементами которого будут модули сумм рядом стоящих комплексных чисел

5. Реализовать в виде модуля набор подпрограмм для выполнения следующих операций над комплексными числами:

- 1) умножения;
- 4) деления;

используя этот модуль, решить задачу - дан массив  $a[m]$  — массив комплексных чисел. получить матрицу  $B[n, m]$ , каждая строка которой получается возведением в степень, равную номеру этой строки, данного массива  $a$ .

6. Реализовать в виде модуля набор подпрограмм для выполнения следующих операций над обыкновенными дробями вида

- 1) сложения;
- 2) вычитания;

используя этот модуль, решить задачу - дан массив  $a$  — массив обыкновенных дробей. найти сумму всех дробей.

7. Реализовать в виде модуля набор подпрограмм для выполнения следующих операций над обыкновенными дробями вида

- 1) умножения;
- 2) деления;

используя этот модуль, решить задачу - дан массив  $a$  — массив обыкновенных дробей. найти произведение всех дробей.

8. Реализовать в виде модуля набор подпрограмм для выполнения следующих операций над обыкновенными дробями вида

- 1) сокращения дроби;
- 2) возведения дроби в степень  $N$  ( $N$  — натуральное);

используя этот модуль, решить задачу - дан массив  $a$  — массив обыкновенных дробей. вычислить среднее арифметическое всех дробей, Результат представить в виде несократимой дроби.

9. Реализовать в виде модуля набор подпрограмм для выполнения следующих операций с квадратными матрицами:

- 1) сложения двух матриц;
- 2) вычисления определителя матрицы ;

10. Реализовать в виде модуля набор подпрограмм для выполнения следующих операций с квадратными матрицами:

- 1) нахождения транспонированной матрицы;
- 2) умножения одной матрицы на другую;

11 Реализовать в виде модуля набор подпрограмм для выполнения следующих операций над векторами:

- 1) сложения;
- 2) вычитания;

12. Реализовать в виде модуля набор подпрограмм для выполнения следующих операций над векторами:

- 1) скалярного умножения векторов;
- 2) умножения вектора на число;

13. Реализовать в виде модуля набор подпрограмм для выполнения следующих операций над векторами:

- 1) нахождения длины вектора.
- 2) сортировки чисел

используя этот модуль, решить задачу - дан массив  $a$  — массив векторов. отсортировать его в порядке убывания длин векторов.

14. Реализовать в виде модуля набор подпрограмм для выполнения следующих операций над числами в 8-миричной системе счисления:

- 1) сложения;
- 2) умножения;

используя этот модуль, решить задачу - возвести число в степень (основание и показатель степени — восьмиричные).

15. Реализовать в виде модуля набор подпрограмм для выполнения следующих операций над числами в 16-миричной системе счисления:

- 1) сложения;
- 2) умножения;

используя этот модуль, решить задачу - возвести число в степень (основание и показатель степени — шестнадцатиричные).

### **Контрольные вопросы:**

- 1) Какие использовали модули для работы вашей программы?
- 2) Какие функции используются для работы вашей программы?

**Домашнее задание** добавьте выбор 3 функции из своего модуля в программу